02-competitive-analysis

R2 — Competitive Speed Analysis of AI CLI Coding Agents

Executive Summary

The operator's goal is for HelixCode to be **3–5x faster than the leading AI CLI coding agents in every situation**. Research into Claude Code, Gemini CLI, Aider, Cline/Roo Code, OpenHands, Cursor, Continue, Plandex and Crush/OpenCode shows that *"agent speed"* decomposes into four largely-independent latency budgets, and the fastest competitors win different budgets:

1. **Cold-start / process latency.** Pure runtime overhead before first interaction. Node.js-based agents (Claude Code, Gemini CLI, Cline) pay ~0.95 s just to boot the V8 runtime; a Go rewrite of Gemini CLI measured **0.014 s startup — 68x faster** — and a 5.6 MB binary vs ~200 MB. HelixCode is already Go: this budget is a *free win* if startup work is kept lazy.

2. **LLM time-to-first-token (TTFT).** Dominated by how much of the prompt prefix is *cached* server-side. Anthropic prompt caching gives **up to 85% latency reduction** on long prompts (a 100K-token example dropped 11.5 s → 2.4 s). Every fast agent is architecturally built *around* keeping the cache prefix stable.

3. **File-edit / apply latency.** Cursor's purpose-built "Fast Apply" model hits **~1000 tok/s** (13x vanilla Llama-3-70B) via *speculative edits*; Morph's specialised 7B apply model reaches **4500–10,500 tok/s at 98% accuracy**. Diff-style edits (Roo Code `apply_diff`, Aider SEARCH/REPLACE) cut output tokens ~30% vs full-file rewrites.

4. **Agent-loop wall-clock.** Parallel tool calls + parallel sub-agents collapse sequential work — Claude Code `/batch` reports **up to 10x** on multi-file refactors.

Highest-impact techniques HelixCode should adopt (detailed table at the end): prompt-cache-stable prefix discipline; persistent on-disk repo-map cache with mtime invalidation (Aider-style); a dedicated fast-apply path (speculative or specialised small model); aggressive tool-call parallelism; lazy/deferred startup; small-model routing for trivial sub-tasks; streaming everywhere.

Per-Competitor Analysis

1. Claude Code (Anthropic CLI)

Runtime / startup. Node.js / TypeScript, distributed via npm; uses the `ink` React-for-terminal renderer (confirmed in vendored `cli_agents/claude-code-source/src/` — `main.tsx`, `ink.ts`). Node runtime boot is the floor cost (~0.9–1 s class, same as Gemini CLI — see §2). `UNCONFIRMED:` Anthropic publishes no official cold-start number.

Context management. No persistent embeddings index. Claude Code relies on *agentic search* — the model issues `Grep`/`Glob`/`Read` tool calls on demand rather than pre-indexing. Context is assembled into a stable prefix (system prompt + tool defs + memory files) plus a growing message list; `/compact` summarises history when the window fills (vendored `src/services/compact/compact.ts`).

LLM-latency techniques. Prompt caching is *the* architectural constraint. The system prompt embeds working directory, platform, shell, OS version and auto-memory paths, so the cache is scoped to one machine+directory; parallel sessions in the same directory share cache. Tool definitions are part of the cached prefix — **adding one MCP tool changes the prefix and invalidates the entire conversation cache**, which is why the tool list is *locked at session start*. The vendored source contains a remarkably detailed `promptCacheBreakDetection.ts` that hashes the system prompt, tool schemas, betas, effort value and `cache_control` scope/TTL separately, and writes a diff of *what broke the cache* — direct evidence that cache-prefix stability is treated as a first-class performance invariant. Supports both the 5-min default and 1-hour extended cache TTL. Streaming is default.

Tool-call parallelism. Claude 4 models do parallel tool use by default (`disable_parallel_tool_use` opt-out). `/batch` decomposes a change into 5–30 independent units, runs each in its own git worktree in parallel — reported up to 10x vs sequential (a 50-file

migration: 100 min → ~15 min). *Programmatic tool calling* lets the model write code that calls tools inside an execution container, reducing round-trips and filtering data before it hits context.

File-edit speed. `Edit` tool uses exact string-match SEARCH/REPLACE on partial file regions (not full-file rewrite). `MultiEdit` batches several edits to one file in a single call.

Caching layers. Server-side prompt cache (Anthropic API); local message/session history on disk; auto-memory (`CLAUDE.md`) loaded into the cached prefix.

Benchmarks. `UNCONFIRMED:` no official latency benchmark; prompt-caching cost data widely reported ($50–100 → $10–19 per long Opus session).

Sources: [1][2][3][4][16][17]

---

2. Gemini CLI (Google)

Runtime / startup. Node.js / TypeScript monorepo (vendored `cli_agents/gemini-cli/packages/` — `cli`, `core`, `a2a-server`, `sdk`). The Node runtime is the dominant cold-start cost: a Go reimplementation ("gmn") measured the official CLI at **0.95 s startup vs 0.014 s for the Go binary — 68x**, and ~200 MB vs 5.6 MB install size; end-to-end with an API response, 10.9 s → 3.2 s (~3.4x). This is the single clearest published data point that **runtime choice alone is a multi-x speed lever** and validates HelixCode's Go foundation.

Context management. `GEMINI.md` persistent context files; a structured context pipeline (vendored `core/src/context/` — `contextManager.ts`, `contextCompressionService.ts`, `memoryContextManager.ts`, `pipeline/contextWorkingBuffer.ts`) with token accounting (`contextTokenCalculator.ts`) and JIT context injection (`tools/jit-context.ts`). Compression service summarises when the window fills.

LLM-latency techniques. Gemini API offers *implicit caching* (automatic, zero setup) and *explicit caching* (developer-controlled `CachedContent`). Managing context at the provider level + a simple in-memory session map yields lower latency (no re-upload of large context) without a vector DB. Streaming default. `UNCONFIRMED:` whether the CLI itself opts into explicit caching by default.

Tool-call parallelism. Supports multiple function calls per turn; `UNCONFIRMED:` exact concurrency policy in the CLI.

File-edit speed. Diff-based edit tools; `UNCONFIRMED:` no published apply-speed number.

Known anti-pattern. Synchronous HTTP handshakes to MCP servers at startup caused ~30 s init in some configs; a stdio-wrapper fix gave ~15x — i.e. *blocking I/O during startup is the enemy*; defer it.

Sources: [5][6][18]

---

3. Aider

Runtime / startup. Python (vendored `cli_agents/aider/aider/`). Python interpreter + heavy imports (tree-sitter, networkx, litellm) make cold start the slowest of the surveyed agents; `UNCONFIRMED`: no official number, but Python import cost is structurally larger than Go.

Context management — the standout technique. Aider's **repository map** is the most-copied context system in the field. `repomap.py` (867 lines, vendored): - Tree-sitter parses every source file into an AST; per-language `tags.scm` query files classify each symbol as *definition* vs *reference* (130+ languages). - A `networkx.MultiDiGraph` connects files via symbol dependencies; **PageRank** ranks symbols by importance, with *personalisation* biasing toward files/identifiers mentioned in the chat (`repomap.py` lines 368–529). - The map is fitted to a token budget (`--map-tokens`, default 1k; multiplied by `map_mul_no_files=8` when no files are in the chat) via binary search with a 15% tolerance — only the highest-ranked symbols survive. - **Persistent on-disk cache:** `diskcache.Cache` in `.aider.tags.cache.v*` (SQLite), keyed by filename, **invalidated by `os.path.getmtime`** — unchanged files are never re-parsed. In-memory `tree_cache`, `tree_context_cache`, `map_cache` layer on top. `refresh="auto"` only rebuilds when inputs change. This is the reference design for *incremental, near-zero-cost* repo context.

LLM-latency techniques. Streaming; via LiteLLM, inherits provider prompt caching (Anthropic/Gemini). Architect/editor split: a strong "architect" model plans, a cheaper/faster "editor" model applies — an early form of **model routing**.

File-edit speed. Many edit formats (vendored `coders/`): `whole`, `diff` (SEARCH/REPLACE blocks), `udiff` (unified diff), `patch`, plus `editor_*` variants. Aider's own benchmarks show diff formats use *far fewer tokens* than whole-file and that unified diffs made GPT-4 Turbo "3X less lazy". The 133-exercise / 225-exercise polyglot benchmarks measure *both* task success and *edit-format compliance* — a model that emits malformed diffs is a speed *and* correctness loss.

Caching layers. On-disk SQLite tags cache; in-memory tree/map caches; provider-side prompt cache through LiteLLM.

Sources: [7][8][13][14][15] + vendored `cli_agents/aider/aider/repomap.py`, `cli_agents/aider/aider/coders/`

---

4. Cline / Roo Code

Runtime / startup. TypeScript VS Code extensions (Cline also ships a standalone mode — vendored `cli_agents/cline/src/standalone/`). Runs in the editor's Node host; startup cost is the extension-host class, not a fresh process. Heavy dependency surface (vendored `package.json`: full OpenTelemetry stack, every provider SDK).

Context management. Documented "context window management": tracks token usage and auto-condenses/truncates as the window fills; no persistent embeddings index by default — relies on explicit `@file`/`@folder` mentions and on-demand reads.

LLM-latency techniques. Provider-agnostic (Anthropic, Bedrock, Vertex, Gemini, Mistral, Cerebras SDKs all vendored); inherits provider prompt caching. A community discussion (`cline/cline#9892`) specifically requests a persistent-session architecture to better exploit Claude Code's prompt cache — i.e. cache exploitation is a recognised gap.

File-edit speed — the key Cline/Roo difference. Cline historically *rewrites the entire file* and shows a full diff view (safer, but high output-token cost). Cline **v3.12** explicitly shipped "faster diff edits", markedly faster apply on large files. **Roo Code's `apply_diff`** outputs *only changed lines* — for a 500-line file with 10 changes, ~10–20 lines vs a full rewrite — independent testing reports **~30% API-cost savings** and faster iteration. Vendored Cline source confirms multiple edit paths: `assistant-message/diff.ts`, `tools/handlers/ApplyPatchHandler.ts`, `prompts/system-prompt/tools/apply_patch.ts`.

Tool-call parallelism. `UNCONFIRMED`: Cline/Roo are largely sequential per turn (plan/act, or Roo's Code/Architect/Ask modes); Roo allows per-mode model selection (a routing lever).

Caching layers. Provider prompt cache; VS Code workspace state; checkpoint/diff history.

Sources: [9][10][1] + vendored `cli_agents/cline/src/`

---

5. OpenHands (formerly OpenDevin)

Runtime / startup. Python; the agent loop runs in a **Docker container** workspace. Container spin-up is a real, non-trivial cold-start cost — the slowest startup profile of the surveyed agents when a fresh sandbox is provisioned.

Architecture. Minimal core: a stateless `Agent` emits `Actions`; a `Conversation` runs the loop and stores an append-only `EventLog`; a `Workspace` (local process or Docker) executes actions and returns `Observations`; the LLM is wrapped by LiteLLM. Event-stream loop: Agent → Action → Environment → Observation → Agent.

Context management. Append-only event log; a *condenser* summarises history to control context growth (cost balloons "easily more without a condenser").

LLM-latency techniques. LiteLLM gives provider prompt caching. `UNCONFIRMED:` no OpenHands-specific latency optimisation beyond the condenser.

Benchmarks. CodeActAgent v1.8 on Claude 3.5 Sonnet: **26% SWE-bench Lite at $1.10 /instance**; current agents 20–45% SWE-bench Verified. Cost-per-task: trivial $0.05–0.30, real SWE-bench fixes $0.50–3, multi-hour runs $5–30. These are *cost/throughput* numbers, not wall-clock latency — OpenHands optimises for autonomy, not interactive speed.

Takeaway for HelixCode. The container sandbox is a correctness/safety asset but a *latency liability*; keep a warm-pool / pre-provisioned sandbox if adopting a similar isolation model.

Sources: [11][12]

---

6. Cursor (CLI / agent aspects)

File-edit speed — the headline technique. Cursor's **"Fast Apply"** turns a terse model-proposed edit into a full merged file. Built on **speculative edits**: a custom speculative-decoding algorithm where *the original file is the draft* — most output tokens are identical to the source, so future tokens are speculated by a *deterministic algorithm*, no draft model needed. A fine-tuned Llama-3-70B ("llama-70b-ft-spec") on Fireworks reaches **>1000 tok/s (~3500 char/s)** — ~13x vanilla Llama-3-70B and ~9x Cursor's prior GPT-4 speculative deployment, "equivalent to a full-file rewrite while up to 9x faster". This is *the* benchmark for file-apply latency and the most directly adoptable single technique.

Context management. Embeddings index of the codebase + AST chunking; agentic retrieval. `UNCONFIRMED:` Cursor's CLI/agent internals are closed-source; most public detail is IDE-side.

LLM-latency techniques. Model routing across a tab/autocomplete model, apply model, and frontier chat/agent models — small/specialised models for narrow tasks.

Sources: [19][20][21]

---

7. Continue

Runtime / startup. TypeScript core, distributed as VS Code/JetBrains extensions and a CLI; in-editor host.

Context management — embeddings-first. Continue *does* build a persistent codebase index: embeddings + keyword search. Embeddings default to **local `transformers.js`**, stored under `~/.continue/index`; metadata in `~/.continue/index/index.sqlite`. AST parsing via tree-sitter, fast text search via **ripgrep**, optional LLM re-ranking of top results. Respects `.gitignore` / `.continueignore`. This is the reference design for the *embeddings* branch of context (vs Aider's *tree-sitter repo-map* branch).

File-edit speed. Apply step; community comparisons note Continue is *slower for "real work"* because it often requires manual "Apply"/"Copy to File" clicks — i.e. human-in-the-loop friction is itself a latency cost. Continue integrates external fast-apply models (e.g. Morph) to close this gap.

Caching layers. Local SQLite index + local embeddings cache; provider prompt cache.

Sources: [9][22] (Morph integration: [23])

---

8. Plandex

Runtime / startup. Go client-server architecture (vendored `cli_agents/plandex/plandex/app/cli/` and `app/server/`). CLI on the dev machine talks to a central Go server with an embedded LiteLLM Python proxy + PostgreSQL + a persistent volume for plan files. Go CLI → fast cold start; the server is a separate process.

Context management. Handles up to ~2M tokens of context directly (~100k/file) and indexes directories of **20M+ tokens via tree-sitter project maps** — top-level symbols (vars/functions/

classes) per file, 30+ languages. “Fast project map generation and syntax validation with tree-sitter.” Building from source needs gcc/g++/make because of the native tree-sitter dependency.

LLM-latency techniques. “Context caching is used across the board for OpenAI, Anthropic and Google models, reducing costs and latency” — i.e. Plandex actively opts into provider prompt caching for all three major providers. Model routing via the embedded LiteLLM proxy.

Takeaway. Plandex is the closest architectural analogue to HelixCode (Go + tree-sitter maps + multi-provider caching + client/server); its scale numbers (20M tokens indexed) prove tree-sitter maps scale far past embeddings.

Sources: [24] + vendored `cli_agents/plandex/`

---

9. Crush / OpenCode (Charm)

Runtime / startup. Go (vendored `cli_agents/crush/`, `go 1.26.2`; internal packages: `agent`, `llm`, `diff`, `diffdetect`, `filetracker`, `lsp`…). Marketed as “combining the speed of Go with Charm’s design… the fastest and most reliable AI CLI coding agent” — i.e. a direct competitor whose explicit pitch is *Go = speed*.

Context management. Integrates **LSP** for real-time code intelligence from actual project files (not just model reasoning); session-based context with multiple simultaneous sessions per project and mid-session model switching with context preserved.

File-edit speed. Dedicated `internal/diff` + `internal/diffdetect` + `internal/filetracker` packages — structured diff application and change tracking.

LLM-latency techniques. Multi-model, switchable mid-session; `UNCONFIRMED:` no published latency number.

Takeaway. Crush validates the thesis that a Go-native, LSP-integrated terminal agent is *the* speed-positioned competitor HelixCode must beat — same language, same LSP idea. HelixCode’s 3–5x edge must come from the *technique stack* (caching, fast-apply, parallelism), not language alone.

Sources: [25] + vendored `cli_agents/crush/`

Cross-Cutting “Techniques Worth Adopting” Table

#	Technique	Best-in-class competitor	What it buys	HelixCode applicability & estimated impact
1	**Prompt-cache-stable prefix discipline** — freeze system prompt + tool defs at session start; never mutate mid-session; hash & diff every cache-break	Claude Code (`promptCacheBreakDetection.ts`)	Up to **85% TTFT reduction**, ~90% input-cost cut on long sessions	**Very high.** Pure architecture. HelixCode mu lock its tool lis at startup and keep a stable cached prefix. Lowest cost / highest payoff
2	**Persistent on-disk repo-map cache with mtime invalidation**	Aider (`diskcache` SQLite, `getmtime`), Plandex (tree-sitter maps, 20M tokens)	Near-zero re-index cost on warm runs; only changed files re-parsed	**Very high.** HelixCode already uses tree-sitter; ad SQLite tag cache keyed b path+mtime + in-memory tre map caches. Directly enables 3–5x c repeated invocations.
3	**Dedicated fast-apply path** — speculative edits (file = draft) or a specialised small apply model	Cursor (~1000 tok/s, 9–13x), Morph (4500–10,500 tok/s, 98%)	File writes go from frontier-model-speed to ~10x faster	**Very high.** Biggest *interactive* speed lever. Adopt speculative-ec decoding for self-hosted models and/or route apply to small specialised model.

#	Technique	Best-in-class competitor	What it buys	HelixCode applicability & estimated impact
4	**Diff-style edits (SEARCH/ REPLACE / apply_diff), not full-file rewrite**	Roo Code `apply_diff`, Aider `diff`/`udiff`	~30% fewer output tokens → proportionally lower latency	**High.** HelixCode's edit format should emit only changed lines; pair wit fuzzy-match application + benchmark edit-format compliance.
5	**Aggressive tool-call parallelism + parallel sub-agents / worktrees**	Claude Code parallel tools + `/batch` (up to 10x)	Collapses sequential I/O and multi-file work	**High.** Go's goroutines make this natural; run independent tool calls and independent file edits concurrently.
6	**Lazy / deferred startup — no blocking I/O before first prompt**	Go-rewrite of Gemini CLI (0.95 s → 0.014 s, 68x); Gemini MCP stdio fix (~15x)	Eliminates cold-start tax; instant first interaction	**High (mostly free).** HelixCode is already Go. Defer MCP handshakes, model discovery, ind loads off the startup path.
7	**Small-model routing for trivial sub-tasks**	Aider (architect/editor split), Cursor (tab/apply/chat models), Plandex (LiteLLM proxy)	Cheap tasks answered by fast small models; frontier model reserved for reasoning	**High.** Route classification/ apply/ summarise to small/local models; respe CONST-036/0 (LLMsVerifier as source of truth for routing metadata).
8	**Provider prompt caching opt-in for**	Plandex (OpenAI + Anthropic + Google)	Latency/cost cut not just for Anthropic	**High.** Wire `cache_control` / `CachedContent` / implicit-cache

#	Technique	Best-in-class competitor	What it buys	HelixCode applicability estimated impact
	ALL providers			for every supported provider, not only Anthropi
9	**Streaming everywhere + token-efficient tool use**	Claude Code, Gemini CLI, all	Perceived latency drops sharply; first token shows immediately	**Medium-high** Stream every model respons and tool-resul rendering; cheap, expected baseline.
10	**History condenser / compaction** to keep the cached prefix small and the window unsaturated	OpenHands condenser, Claude Code `/compact`, Gemini compression service	Prevents context-bloat slowdowns and cost blow-ups in long sessions	**Medium.** Needed for long autonomous runs; seconda to 1–8 for interactive speed.
11	**LSP integration for context** instead of pure model reasoning	Crush, (Cursor IDE)	Accurate symbol context without round-tripping the model	**Medium.** Improves quality and reduces waste retrieval turns complements the repo map.
12	**Warm sandbox pool** if container isolation is used	(anti-pattern lesson from OpenHands)	Removes container-provision cold start	**Medium / conditional.** Only if HelixCode run containerised workspaces — keep a pre-warmed pool.

Sources / Citations

1. Cline — *Enable Prompt Caching for Claude Code Provider (Persistent Session Architecture)*, GitHub Discussion #9892 — https://github.com/cline/cline/discussions/9892

2. *How Prompt Caching Actually Works in Claude Code* — https://www.claudecodecamp.com/p/how-prompt-caching-actually-works-in-claude-code
3. Anthropic — *How Claude Code uses prompt caching* (Claude Code Docs) — https://code.claude.com/docs/en/prompt-caching
4. Introl — *Claude Code CLI: The Definitive Technical Reference* — https://introl.com/blog/claude-code-cli-comprehensive-guide-2025
5. Tanaike — *Accelerating Gemini CLI: A Node.js Wrapper for Google Apps Script MCP Servers* — https://medium.com/google-cloud/accelerating-gemini-cli-a-node-js-wrapper-for-google-apps-script-mcp-servers-a4295283d2ca
6. Google — *Gemini CLI* (official docs) — https://google-gemini.github.io/gemini-cli/
7. Aider — *Building a better repository map with tree sitter* — https://aider.chat/2023/10/22/repomap.html
8. Aider — *Repository map* (docs) — https://aider.chat/docs/repomap.html
9. DevToolReviews — *Cline vs Roo Code vs Continue (2026)* — https://www.devtoolreviews.com/reviews/cline-vs-roo-code-vs-continue
10. Cline — *Cline v3.12: Faster Diff Edits, Model Favorites, and More* — https://cline.bot/blog/cline-v3-12-faster-diff-edits-model-favorites-and-more
11. mgx.dev — *A Comprehensive Analysis of OpenDevin (OpenHands): Architecture, Development, Use Cases, and Challenges* — https://mgx.dev/insights/a-comprehensive-analysis-of-opendevin-openhands-architecture-development-use-cases-and-challenges/62fee7b52567490da851f0ed7cb2bf9f
12. OpenHands — *OpenHands: An Open Platform for AI Software Developers as Generalist Agents* (arXiv) — https://arxiv.org/pdf/2407.16741
13. Aider — *Code editing leaderboard* — https://aider.chat/docs/leaderboards/edit.html
14. Aider — *Unified diffs make GPT-4 Turbo 3X less lazy* — https://aider.chat/docs/unified-diffs.html
15. Aider — *Edit formats* — https://aider.chat/docs/more/edit-formats.html
16. Anthropic — *Prompt caching* (Claude API docs) — https://platform.claude.com/docs/en/build-with-claude/prompt-caching
17. Anthropic — *Parallel tool use* (Claude API docs) — https://platform.claude.com/docs/en/agents-and-tools/tool-use/parallel-tool-use
18. Owada Tomohiro — *I Rewrote Google's Gemini CLI in Go — 68x Faster Startup* (DEV) — https://dev.to/owada_tomohiro_28ec22f5ee/i-rewrote-googles-gemini-cli-in-go-68x-faster-startup-30em
19. Cursor — *Editing Files at 1000 Tokens per Second* — https://cursor.com/blog/instant-apply
20. Fireworks — *How Cursor built Fast Apply using the Speculative Decoding API* — https://fireworks.ai/blog/cursor

21. Bind AI — *How Cursor AI Implemented Instant Apply: File Editing at 1000 Tokens per Second* — https://blog.getbind.co/2024/10/02/how-cursor-ai-implemented-instant-apply-file-editing-at-1000-tokens-per-second/
22. Continue — *How to Set Up @Codebase Context Provider in Continue* — https://docs.continue.dev/customize/context/codebase
23. Morph — *Fast Apply: Code Merging at 10,500 tok/s for AI Agents* — https://www.morphllm.com/fast-apply-model
24. Plandex — GitHub repository + *Context Management* docs — https://github.com/plandex-ai/plandex and https://docs.plandex.ai/core-concepts/context-management/
25. Charm — *Crush* (GitHub) — https://github.com/charmbracelet/crush
26. Anthropic — *Extended 1-hour prompt-cache TTL announcement* (X) — https://x.com/AnthropicAI/status/1925633128174899453

Vendored sources inspected (in-repo, `/run/media/milosvasic/DATA4TB/Projects/HelixCode/cli_agents/`): `aider/aider/repomap.py` (867 lines — tree-sitter + networkx PageRank + diskcache SQLite + mtime invalidation), `aider/aider/coders/` (whole/diff/udiff/patch/editor edit formats), `claude-code-source/src/services/api/promptCacheBreakDetection.ts` (cache-prefix hashing + break diff), `claude-code-source/src/` (`main.tsx`, `ink.ts` — Node/React-ink runtime; `services/compact/compact.ts`), `gemini-cli/packages/core/src/context/` (contextManager, contextCompressionService, pipeline, jit-context), `cline/src/core/assistant-message/diff.ts`, `cline/src/core/task/tools/handlers/ApplyPatchHandler.ts`, `plandex/plandex/app/cli/` + `app/server/` (Go client/server), `crush/` (`go.mod` go 1.26.2; `internal/{agent,llm,diff,diffdetect,filetracker,lsp}`).